

Getting started with PyLucene - Retrieval

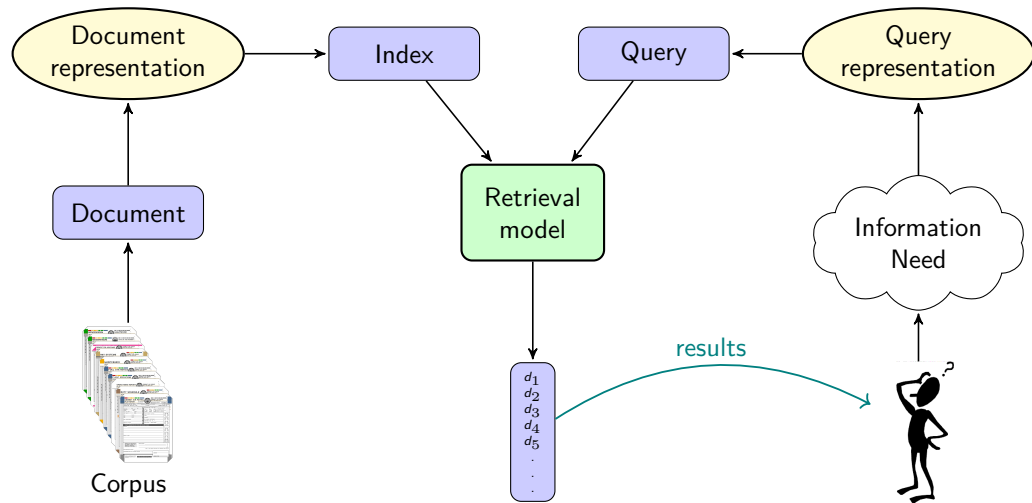
Introduction to Information Retrieval

Dwaipayan Roy

Department of Computational and Data Sciences



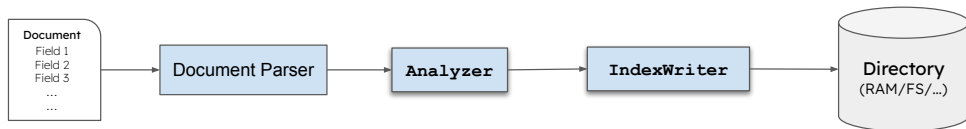
Information Retrieval - A Graphical Representation



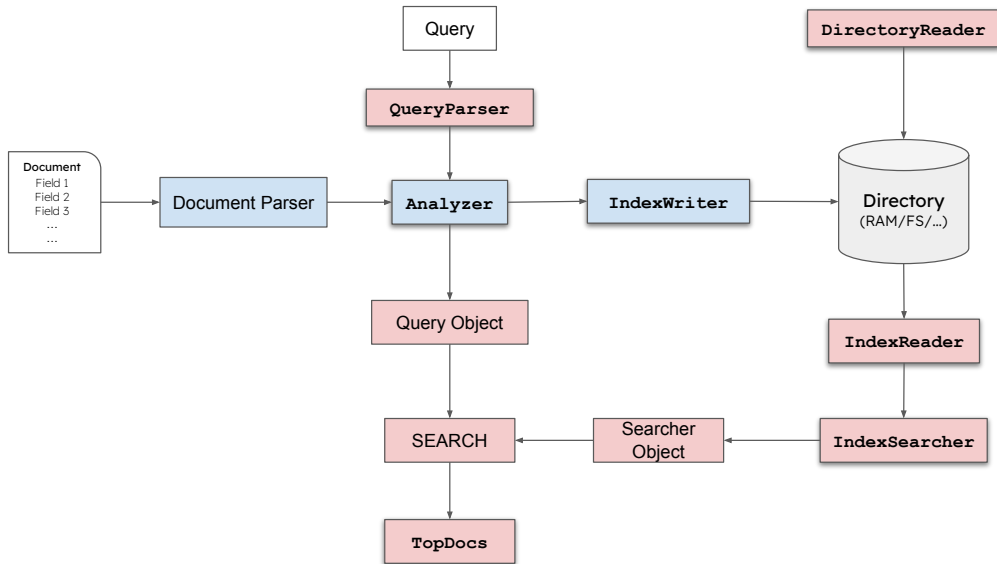
Information Retrieval using Lucene

- 1 Indexing a collection.
- 2 Searching in a collection.

Indexing a Collection in Lucene



Searching in Lucene



Searching in Lucene

- Input: A query (string) (along with language, stopwords list)

Searching in Lucene

- Input: A query (string) (along with language, stopwords list)
 - ▶ `index_path` = path to Lucene index to be searched;
 - ▶ `search_field` = searching field;
 - ▶ `top_size` = desired number of documents to retrieve.
- Output: A list of document (TopDocs).

Searching in Lucene

- Input: A query (string) (along with language, stopwords list)
 - ▶ `index_path` = path to Lucene index to be searched;
 - ▶ `search_field` = searching field;
 - ▶ `top_size` = desired number of documents to retrieve.
- Output: A list of document (TopDocs).
- **Note** - need to use the same analyzer employed while indexing.

Searching in Lucene: Creating an Analyzer

- Set the same analyzer that has been used while indexing;

```
from org.apache.lucene.analysis.en import EnglishAnalyzer

def makeSearch(index_path, q):
    analyzer = EnglishAnalyzer()
```

Searching in Lucene: `QueryParser`

- Parse (analyze) the query string, to be searched in the `search_field`

```
from org.apache.lucene.queryparser.classic import QueryParser
from org.apache.lucene.analysis.en import EnglishAnalyzer

def makeSearch(index_path, q):
    analyzer = EnglishAnalyzer()
    query = QueryParser("PLOT", analyzer).parse(q)
```

Searching in Lucene: Creating an `IndexSearcher`

- Open the Lucene index for search;

```
from org.apache.lucene.search import IndexSearcher
from org.apache.lucene.index import DirectoryReader
from org.apache.lucene.store import FSDirectory
from java.io import File

def makeSearch(index_path, q):
    analyzer = StandardAnalyzer()
    directory = FSDirectory.open(File(index_path).toPath())
    searcher = IndexSearcher(DirectoryReader.open(directory))
```

Searching in Lucene: Setting Similarity Function

```
from org.apache.lucene.search.similarities import BM25Similarity
from org.apache.lucene.search.similarities import LMJelinekMercerSimilarity
from org.apache.lucene.search.similarities import LMDirichletSimilarity

def makeSearch(index_path, q):
    ...
    searcher.setSimilarity(BM25Similarity(k1, b))
    searcher.setSimilarity(LMJelinekMercerSimilarity(LAMBDA))
    searcher.setSimilarity(LMDirichletSimilarity(MU))
    ...
```

Searching in Lucene: performing the search()

```
def makeSearch(index_path, q):  
    ...  
    scoreDocs = searcher.search(query, 50).scoreDocs  
    ...
```

Searching in Lucene: accessing individual retrieved documents

```
def makeSearch(index_path, q):  
    ...  
    for scoreDoc in scoreDocs:  
        doc = searcher.storedFields().document(scoreDoc.doc)  
        print (doc.get("TITLE"), 'name:', doc.get("PLOT"))
```

Searching in Lucene: putting everything together

```
def makeSearch(index_path, q, field):
    print ("Searching for:", q)

    analyzer = EnglishAnalyzer()
    directory = FSDirectory.open(File(index_path).toPath())
    searcher = IndexSearcher(DirectoryReader.open(directory))
    searcher.setSimilarity(BM25Similarity(1.2, 0.75))

    query = QueryParser(field, analyzer).parse(q)

    print(query)

    scoreDocs = searcher.search(query, 10).scoreDocs

    print("{} total matching documents".format(len(scoreDocs)))

    for scoreDoc in scoreDocs:
        doc = searcher.storedFields().document(scoreDoc.doc)
        print (doc.get("TITLE"), scoreDoc.score)
```

- Input: A query (string).

Searching in Lucene: putting everything together

```
def makeSearch(index_path, q, field):
    print ("Searching for:", q)

    analyzer = EnglishAnalyzer()
    directory = FSDirectory.open(File(index_path).toPath())
    searcher = IndexSearcher(DirectoryReader.open(directory))
    searcher.setSimilarity(BM25Similarity(1.2, 0.75))

    query = QueryParser(field, analyzer).parse(q)

    print(query)

    scoreDocs = searcher.search(query, 10).scoreDocs

    print("{} total matching documents".format(len(scoreDocs)))

    for scoreDoc in scoreDocs:
        doc = searcher.storedFields().document(scoreDoc.doc)
        print (doc.get("TITLE"), scoreDoc.score)
```

- Input: A query (string).
 - ▶ `index_path` = where Lucene index stored;
 - ▶ `search_field` = searching field;
 - ▶ `top_size` = desired number of documents to retrieve;
 - ▶ `parameters` of the retrieval model;
 - ▶ `field to retrieve`.
- Output: A list of documents.

Searching in Lucene: Variety of Query Classes

- **TermQuery**: Exact matching of a single term in a field (e.g., search for genre "Comedy"). ([Link](#))
- **PhraseQuery**: Exact matching of sequence of terms in field (e.g., search for "dream sharing" in plot). ([Link](#))
- **TermRangeQuery**: Range queries on text fields (e.g., movies with titles between "A" and "M"). ([Link](#))
- **Numeric Range Query**: Range queries on numeric fields (e.g., movies from year 2000 to 2010). ([Link](#))
- **PrefixQuery**: Matches terms starting with a prefix (e.g., titles starting with "The"). ([Link](#))
- **BooleanQuery**: Combines multiple queries with Boolean logic (e.g., genre "Drama" AND year 1994). ([Link](#))
- **WildcardQuery**: Matches terms using wildcard patterns (e.g., title with pattern "Th"). ([Link](#))
- **FuzzyQuery**: Matches terms similar to the specified term (e.g., director name "Tarentino" for "Tarantino"). ([Link](#))
- **MatchAllDocsQuery**: Matches all documents in the index (e.g., retrieve all movies). ([Link](#))

https://lucene.apache.org/core/10_0_0/core/org/apache/lucene/search/package-summary.html

TermQuery

```
TermQuery (Term("origin", "Bollywood"))
```

- Exact matching: searches for exact term "Bollywood" as-is
- No analysis - the term is not processed through the analyzer
- Case-sensitive - depends on how the field was originally indexed

QueryParser

```
QueryParser("origin", analyzer).parse("Bollywood")
```

- Analyzed search - search term goes through the analyzer (e.g., StandardAnalyzer).
- May be transformed - could be lowercased, stemmed, or processed based on analyzer rules
- More flexible - can parse complex query syntax like "Bollywood AND Drama"

Take Away

- If `analyzer` lowercased during indexing: `TermQuery` may find **nothing**, `QueryParser` (applied with the same `analyzer`) will find matches.
- If indexed in `StringField`: `TermQuery` is applicable; `QueryParser` will **not** find anything.

TermQuery

```
TermQuery (Term("origin", "Bollywood"))
```

- Exact matching: searches for exact term "Bollywood" as-is
- No analysis - the search term goes through the analyzer
- Case-sensitive - the search term was originally in the index

QueryParser

```
QueryParser("origin", analyzer).parse("Bollywood")
```

- Analyzed search - search term goes through the analyzer (e.g., StandardAnalyzer).

- TermQuery** → for exact matching of the stored term.

- QueryParser** → the same analysis to search terms was applied during indexing.

...d be lowercased,
...ed on analyzer rules
...omplex query syntax
ma"

Take Away

- If **analyzer** lowercased during indexing: **TermQuery** may find **nothing**, **QueryParser** (applied with the same **analyzer**) will find matches.
- If indexed in **StringField**: **TermQuery** is applicable; **QueryParser** will **not** find anything.